

TRABAJO PRÁCTICO INTEGRADOR

Metodología de Sistemas II

Body Paint — Sistema E-Commerce

Gestión Scrum · 3 Sprints · Rol Vendedor

INSTITUCIÓN	UTN — Facultad Regional Villa María (FRVM)
CARRERA	Tecnicatura Universitaria en Programación
CÁTEDRA	Metodología de Sistemas II · 2026
PRODUCT OWNER	Prof. Daniela Córdoba
GRUPO	N° 9
SCRUM MASTER	Genaro Molina
DEVELOPMENT TEAM	Paulo Zito · Maxi Zampa

Repositorio y Gestión del Proyecto

GitHub:
<https://github.com/Paulozzt/TPI-metodologia2-2026-g9/tree/main>

1. Roles del Sistema y Responsabilidades

ROL: CLIENTE

- Registrarse en la plataforma con datos personales y dirección de envío.
- Explorar el catálogo y agregar/quitar productos del carro de compras.
- Seleccionar domicilio de envío y método de pago al confirmar el pedido.
- Ingresar códigos de cupón de descuento en el carro antes de confirmar.
- Realizar reclamos dentro de los 5 días hábiles posteriores a la entrega.

ROL: VENDEDOR

- Visualizar el estado de todos los pedidos pendientes de entrega.
- Generar y administrar cupones de descuento para campañas personalizadas.
- Asignar cupones a clientes específicos y notificarlos por email automáticamente.
- Generar reportes: productos más vendidos, zonas con más pedidos, top 10 clientes.
- Cancelar entregas ante problemas de stock o logísticos.
- Notificar al proveedor de envío cuando un pedido está listo para retiro.

ROL: ADMINISTRADOR DE PRODUCTOS

- Visualizar y gestionar el catálogo completo de productos simples y kits.
- Crear, editar y dar de baja productos y kits del sistema.
- Configurar el umbral de stock mínimo por producto (alerta visual en rojo).
- Realizar carga masiva de productos/precios mediante archivos CSV o XLS.
- Acceder a reportería interna: productos en kits, productos cercanos al stock mínimo.

2. Visión del Producto y Stack Tecnológico

Visión

Desarrollar una plataforma web eficiente y escalable que centralice el proceso de venta minorista y el control de inventario para **Body Paint**, permitiendo a los clientes comprar de forma intuitiva mientras los vendedores gestionan promociones, entregas y analizan métricas comerciales en tiempo real.

Objetivos Estratégicos

- **Experiencia de Usuario:** carrito intuitivo con persistencia de datos en sesión y aplicación de cupones validada en tiempo real.
- **Gestión Inteligente:** alertas visuales automáticas (resaltado rojo + icóno) cuando el stock alcanza el umbral mínimo configurable.
- **Escalabilidad:** arquitectura de componentes React desacoplados, preparada para integrar backend REST (Spring Boot) en iteraciones futuras.
- **Analítica Comercial:** módulo de reportes con visualizaciones animadas de productos más vendidos, zonas geográficas y ranking de clientes.

Stack Tecnológico

Capa	Tecnología	Justificación
Frontend	React 19 + Vite + TypeScript + Tailwind CSS 4	Velocidad de desarrollo, componentes reutilizables, tipado estático.
Animaciones	Framer Motion (motion/react)	Transiciones fluidas en listas de cupones y gráficos de reportes.
Iconografía	Lucide React	Librería consistente, tree-shakeable, alineada con el diseño.
Estado	React useState (in-memory)	Persistencia de sesión; desacoplado para futura integración con API REST.
Bundler	Vite 6	HMR ultrarrápido; build optimizado para Vercel/Netlify.
Despliegue	Vercel / Netlify	CI/CD automático desde rama main en GitHub.
Backend (plan)	Java 17 + Spring Boot 3 + MySQL 8	Robustez y manejo de transacciones — planificado para iteraciones futuras.

Marco de Trabajo — Scrum

El equipo adoptó un flujo Scrum con Sprints de 2 semanas. Ceremonias definidas:

- **Sprint Planning:** selección y estimación de User Stories del Product Backlog junto al PO.
- **Daily Sync (WhatsApp):** reporte diario: ¿Qué hice ayer? / ¿Qué haré hoy? / ¿Tengo impedimentos?
- **Weekly Sync (Presencial/Meet):** revisión de código, integración de ramas y resolución de bloqueos.
- **Sprint Review:** demostración del incremento funcional a la Prof. Córdoba.
- **Retrospectiva:** análisis de lo que funcionó, lo que no, y acciones de mejora.

Definition of Done (DoD)

- Funcionalidad cumple el 100 % de los Criterios de Aceptación definidos y aprobación del PO.
- Peer Review: ningún código llega a **main** sin revisión de al menos un integrante.
- Build exitoso sin errores de compilación (TypeScript strict mode).
- Interfaz 100 % responsiva, validada en mobile y escritorio.
- Documentación del sprint actualizada en el PDF del proyecto.

SPRINT 1 · 11 may – 25 may 2026 · Setup, MVP Parcial & US1 Vendedor

1. Objetivo del Sprint

Establecer la arquitectura base del proyecto (React + Vite + TypeScript + Tailwind), modelar los tipos de dominio core (*Product, Client, Coupon, Order, EmailLog*), implementar el MVP parcial del rol Cliente (registro y visualización de catálogo) y del Administrador de Productos (visualización y configuración de stock mínimo), e implementar la primera User Story del rol Vendedor: generación y gestión de cupones.

2. User Stories del Sprint Backlog

ID	Descripción	Rol	Estado
MVP-1	Registrar cliente (modal con datos personales + región)	Cliente	✓ Completo
MVP-2	Visualizar catálogo con alerta visual de stock	Cliente / Admin	✓ Completo
MVP-3	Configurar stock mínimo por producto	Admin Productos	✓ Completo
MVP-4	Crear producto (mockData inicial en memoria)	Admin Productos	✓ Completo
US-1	Generación de cupones de descuento	Vendedor	✓ Completo

3. Detalle — US-1: Generación de Cupones de Descuento

US-1 · Vendedor	Como vendedor, quiero generar cupones de descuento para ofrecer promociones.
Criterios de Aceptación	- El vendedor puede ingresar desde el panel: código de cupón (texto único), validez desde/hasta, productos abarcados (selección múltiple de simples y kits) y monto de descuento en pesos. - El sistema valida que el código de cupón no esté duplicado antes de guardarlo. - El cupón creado queda en estado "activo" por defecto y es visible en la lista del panel. - El vendedor puede activar/desactivar un cupón existente sin eliminarlo. - El vendedor puede eliminar un cupón que aún no fue utilizado. - Si no se seleccionan productos específicos, el cupón aplica a todo el catálogo.
Estimación	Esfuerzo: Medio Complejidad: Media Incertidumbre: Baja
Justificación	El esfuerzo es medio porque requiere modelar la entidad Coupon con sus relaciones (productos abarcados, validez, estado) y crear CouponCreator.tsx con formulario validado y lista de cupones con acciones. La complejidad es media al manejar selección múltiple de productos/kits y validación de fechas. La incertidumbre es baja: todas las variables del cupón están explícitamente definidas en el enunciado del dominio.

4. Resultado del Sprint

✓ Funcionalidades Implementadas	■ Pospuesto / Pendiente
• Arquitectura React + Vite + TypeScript configurada (tsconfig strict, paths alias @/). • types.ts: interfaces Product, Client, Coupon, Order, OrderItem, EmailLog. • mockData.ts: 12 productos (simples y kits), 8 clientes con regiones, 3 cupones iniciales, 6 órdenes históricas. • App.tsx: navegación por portales Vendedor/Cliente con paletas de colores intercambiable. • Panel Admin — Tabla de stock con alerta visual roja + icóno AlertTriangle bajo mínimo. • Panel Admin — Editor inline de stock mínimo configurable por producto. • Modal de Registro de Cliente con validación de campos obligatorios. • CouponCreator.tsx: formulario completo con selector de productos/kits. • Lista de cupones con acciones toggle activo/inactivo y eliminación.	• Backend (Spring Boot + MySQL) — pospuesto a iteraciones futuras. • Confirmación de pedido completa (carro de compras) — pospuesto al Sprint 2. • Envío de emails reales (SMTP) — simulado en memoria en esta iteración.

5. Decisiones Técnicas Relevantes

- **Arquitectura 100 % frontend:** se decidió empezar con datos en memoria (mockData.ts) para iterar rápidamente sin dependencia de backend. La lógica de negocio en validateCoupon() y simulateSendEmail() en helpers.ts permite extraerla a un servicio REST sin modificar los componentes.
- **Sistema de paletas de colores dinámico:** switcher de 5 paletas de marca vía CSS custom properties (--brand-primary, etc.) con persistencia en localStorage. Permite adaptar la identidad visual sin reescribir clases Tailwind.
- **Modelo de dominio tipado:** TypeScript strict mode desde el inicio garantiza coherencia entre mockData, componentes y helpers. Cada entidad tiene su interface en types.ts compartida globalmente.
- **Gitflow simplificado:** rama main protegida, feature branches por US (feature/us1-coupon-creator), PR con peer review obligatorio antes de merge.

SPRINT 2 · 25 may – 02 jun 2026 · MVP Completo & US2 Vendedor

1. Objetivo del Sprint

Completar el MVP obligatorio de la cátedra — carro de compras funcional, confirmación de pedido con pago en efectivo contra entrega y visualización de estado de pedidos para el Vendedor — e implementar la segunda User Story del rol Vendedor: asignación de cupones a clientes específicos con notificación simulada por email.

2. User Stories del Sprint Backlog

ID	Descripción	Rol	Estado
MVP-5	Armar carro de compras (agregar/quitar, ver total)	Cliente	✓ Completo
MVP-6	Confirmar pedido con pago en efectivo contra entrega	Cliente	✓ Completo
MVP-7	Visualizar estado de pedidos (panel del vendedor)	Vendedor	✓ Completo
US-2	Asignación de cupones a clientes específicos + notif. email	Vendedor	✓ Completo

3. Detalle — US-2: Asignación de Cupones a Clientes

US-2 · Vendedor	Como vendedor, quiero asignar cupones a clientes específicos para campañas personalizadas.
Criterios de Aceptación	• El vendedor selecciona un cupón activo del panel y elige uno o más clientes destinatarios. • El sistema permite filtrar clientes por región geográfica y buscar por nombre o email. • Al confirmar la asignación, el sistema genera un email simulado a cada cliente con: código, descuento, validez y productos abarcados. • El panel muestra el log de emails enviados con destinatario, asunto y timestamp. • El sistema previene la asignación duplicada: si el cliente ya tiene el cupón, lo informa y no reenvía. • Al confirmarse el pago de un pedido con cupón aplicado, el sistema marca automáticamente el cupón como "usado" para ese cliente (usedByClientId). • El cupón marcado como usado ya no puede ser aplicado nuevamente por el mismo cliente.
Estimación	Esfuerzo: Alto Complejidad: Alta Incertidumbre: Media
Justificación	El esfuerzo es alto porque requiere integrar CouponAssigner.tsx con el estado global de coupons y clients, implementar lógica de asignación con deduplicación (Set para evitar duplicados) y mantener consistencia entre assignedClientId y usedByClientId al confirmar un pago en handleOrderPaid(). La complejidad es alta por la concurrencia de estados: el cupón debe actualizarse en tiempo real tanto en el panel del vendedor como en el simulador del cliente. La incertidumbre es media porque la integración con un servicio SMTP real puede presentar desafíos de configuración no anticipados en la transición frontend→backend.

3b. Flujo de validación de cupones (helpers.ts)

La función validateCoupon() implementa todas las reglas de negocio del dominio:

- Verifica que el cupón esté activo.
- Verifica que la fecha actual esté dentro del rango de validez ($\text{validityStart} \leq \text{hoy} \leq \text{validityEnd}$).
- Verifica que el cliente esté en la lista de assignedClientIds (si la lista no está vacía).
- Verifica que el cliente no haya usado ya el cupón (usedByClientIds).
- Verifica que el subtotal del carrito sea mayor al monto del descuento.
- Verifica que el carrito contenga al menos un producto abarcado (si applicableProductIds no está vacío).

Regla de Negocio Clave

El descuento solo se aplica si el monto del pedido es mayor al descuento obtenido. Implementado en helpers.ts: `if (cartSubtotal <= coupon.discountAmount) → isValid: false.`

4. Resultado del Sprint

✓ Funcionalidades Implementadas	■ Pospuesto / Pendiente
• InteractiveSimulator.tsx: catálogo con carrito (agregar/quitar/actualizar cantidad). • Validación de stock en tiempo real: no permite agregar más unidades que el stock disponible. • Formulario de confirmación: selector de cliente, domicilio de envío, 3 métodos de pago. • Actualización automática de stock al confirmar una compra (handleOrderCreated). • Panel del Vendedor — Pestaña Pedidos: lista de todos los pedidos con estado pending/paid/cancelled. • Confirmación de pago: marca el pedido como "paid" y el cupón como "usado". • CouponAssigner.tsx: selector de cupón, filtro por región, búsqueda por nombre/email, selección múltiple. • Log de emails simulados: registro visible con asunto, destinatario y timestamp. • MVP completo: las 6 funcionalidades mínimas de la cátedra están operativas.	• Integración con API de Mercado Pago real — simulada en frontend. • Envío de emails reales (SMTP/SendGrid) — pendiente hasta integración con backend. • Módulo de reclamos post-entrega — fuera del alcance del MVP.

5. Decisiones Técnicas Relevantes

- **Estado compartido elevado (lifting state up):** products, clients, coupons, orders y emailLogs se mantienen en App.tsx y se pasan como props + callbacks. Garantiza coherencia entre el portal Vendedor y el portal Cliente sin necesidad de un store global.
- **Simulación SMTP en helpers.ts:** simulateSendEmail() genera un EmailLog con subject, body y timestamp reales, replicando la estructura de un email transaccional — facilita la migración futura a SendGrid o Nodemailer.
- **Animaciones con Framer Motion:** AnimatePresence + motion.div en la lista de cupones produce entradas/salidas fluidas (opacity + scale) al crear o eliminar, mejorando la percepción de respuesta del sistema.
- **Prevención de duplicados con Set:** Array.from(new Set([...assignedClientIds, ...newIds])) garantiza idempotencia, evitando envíos duplicados en asignaciones parciales.

1. Objetivo del Sprint

Implementar la tercera y última User Story del rol Vendedor: el módulo de reportes y análisis de ventas con tres vistas analíticas (productos más vendidos, zonas geográficas con más pedidos y ranking top 10 de clientes). Completar el pulido visual de la aplicación e implementar el sistema de paletas de colores intercambiable. Preparar la demo final para la presentación del 8 de junio de 2026.

2. User Stories del Sprint Backlog

ID	Descripción	Rol	Estado
US-3	Reportes y análisis de ventas (3 reportes)	Vendedor	✓ Completo
TECH-1	Sistema de paletas de colores dinámico (5 temas)	Transversal	✓ Completo
TECH-2	Pulido UI/UX: footer de estado, responsive mobile	Transversal	✓ Completo
TECH-3	Revisión de mockData y casos edge (stock=0, vencidos)	QA	✓ Completo

3. Detalle — US-3: Reportes y Análisis de Ventas

US-3 · Vendedor	Como vendedor, quiero ver reportes de ventas para analizar el negocio.		
Criterios de Aceptación	- Reporte 1 — Productos más vendidos: tabla ordenada de mayor a menor por unidades totales vendidas, mostrando nombre, categoría, unidades vendidas y revenue generado. - Reporte 2 — Zonas con más pedidos: gráfico de barras horizontales animadas por región geográfica, mostrando cantidad de pedidos y revenue por zona. - Reporte 3 — Top 10 Clientes: ranking con podio visual para los 3 primeros y tabla completa con nombre, email, cantidad de compras y monto total invertido. - Los tres reportes se calculan dinámicamente sobre el estado de orders; se actualizan automáticamente al confirmarse nuevos pagos desde el simulador. - KPIs globales visibles al tope del módulo: ventas confirmadas, revenue total, cupones activos y clientes registrados.		
Estimación	Esfuerzo: Alto	Complejidad: Media	Incertidumbre: Baja
Justificación	El esfuerzo es alto porque requiere implementar tres algoritmos de agregación distintos: agrupación por productId (reporte 1), por region (reporte 2) y por clientId con suma de totales (reporte 3). La complejidad lógica es media ya que los algoritmos son reducción/grupación estándar sobre arrays. La incertidumbre es baja porque el alcance de los tres reportes está completamente especificado en el enunciado del dominio, sin ambigüedades.		

3b. Algoritmos de Agregación — SalesReports.tsx

Los tres reportes se calculan sobre el array de órdenes filtrado por status="paid":

- **Reporte 1 (productos más vendidos):** itera sobre order.items acumulando quantity y revenue (quantity × unitPrice) agrupados por productId en un Map. Resultado ordenado descendente por unidades.
- **Reporte 2 (zonas):** agrupa por order.region sumando orderCount y revenue. Las barras se animan con Framer Motion (width de 0 % a ratio×100 %) para feedback visual inmediato.
- **Reporte 3 (top 10 clientes):** agrupa por order.clientId sumando order.total y contando órdenes. El podio visual distingue 1°, 2° y 3° lugar con estilos diferenciados.

Reactividad de los Reportes

Los datos son reactivos: al confirmar un pago en el InteractiveSimulator, handleOrderPaid() actualiza el estado en App.tsx, lo que dispara un re-render de SalesReports con los valores recalculados automáticamente — sin necesidad de un botón "Actualizar".

4. Resultado del Sprint

✓ Funcionalidades Implementadas

- SalesReports.tsx: tres reportes analíticos completamente funcionales y reactivos.
- KPI cards: revenue total, ventas confirmadas, cupones activos, clientes registrados.
- Reporte 1: tabla de productos más vendidos con unidades y revenue.
- Reporte 2: barras horizontales animadas de pedidos por zona geográfica.
- Reporte 3: podio visual top 3 + tabla top 10 clientes con monto invertido.
- Sistema de paletas: 5 temas intercambiables vía CSS custom properties + localStorage.
- Footer de estado del sistema (SISTEMA ONLINE · MERCADO PAGO: CONECTADO).
- Casos edge revisados: stock=0 bloquea botón, cupón vencido rechazado con mensaje.
- Aplicación lista para demo: todos los flujos testeados manualmente end-to-end.

■ Pospuesto / Pendiente

- Backend REST (Spring Boot + MySQL) — deuda técnica para iteración futura.
- Emails reales (SMTP) — requiere backend; simulado satisfactoriamente.
- Módulo de reclamos — fuera del alcance del MVP.
- Carga masiva CSV/XLS — US futura, no priorizada en este sprint.

5. Decisiones Técnicas Relevantes

- **Cálculo de reportes por derivación reactiva:** los datos se derivan del array orders en cada render. Para el volumen del mockData es eficiente; en producción migrarían a consultas SQL agrupadas (GROUP BY product_id ORDER BY SUM(quantity) DESC).
- **Animaciones de barras con Framer Motion:** motion.div con initial={{ width: 0 }} y animate={{ width: ratio×100% }} produce efecto de llenado progresivo al montar el componente, mejorando la legibilidad del reporte geográfico.
- **Podio visual diferenciado:** el cliente campeón (1°) tiene card de mayor tamaño, fondo amber y badge "CAMPEÓN DE COMPRAS" con animación bounce. Los puestos 2° y 3° usan estilos más discretos.
- **Preparación para la presentación final:** la aplicación fue revisada en mobile y escritorio. Se verificaron todos los flujos: registro → carrito → cupón → confirmación → actualización de reportes.

4. Product Backlog Completo

Backlog identificado a partir del análisis del dominio completo. Las US marcadas como implementadas corresponden al alcance del MVP + las 3 US del rol Vendedor desarrolladas en los 3 sprints.

ID	User Story	Rol	Prior.	Estado
MVP-1	Registrar cliente	Cliente	Alta	✓ Implementado
MVP-2	Visualizar catálogo y alerta de stock	Cliente/Admin	Alta	✓ Implementado
MVP-3	Armar carro de compras	Cliente	Alta	✓ Implementado
MVP-4	Confirmar pedido (efectivo contra entrega)	Cliente	Alta	✓ Implementado
MVP-5	Visualizar productos (Admin)	Admin	Alta	✓ Implementado
MVP-6	Crear producto (Admin)	Admin	Alta	✓ Implementado
MVP-7	Visualizar estado de pedidos	Vendedor	Alta	✓ Implementado
US-1	Generar cupones de descuento	Vendedor	Alta	✓ Implementado
US-2	Asignar cupones a clientes + email	Vendedor	Alta	✓ Implementado
US-3	Reportes de ventas (3 reportes)	Vendedor	Alta	✓ Implementado
US-4	Confirmar pedido con tarjeta de crédito	Cliente	Media	Backlog
US-5	Confirmar pedido con Mercado Pago	Cliente	Media	Backlog
US-6	Gestión de reclamos (nueva entrega / reembolso)	Cliente/Vend.	Media	Backlog
US-7	Carga masiva de productos (CSV/XLS)	Admin	Media	Backlog
US-8	Tracking de envío (número de seguimiento)	Cliente/Vend.	Media	Backlog
US-9	Armar y gestionar kits de productos	Admin	Media	Backlog
US-10	Dar de baja productos/kits	Admin	Baja	Backlog
US-11	Cupón aplicado por el cliente en el carrito	Cliente	Media	✓ Implementado
US-12	Reporte: productos en kits / cercanos a stock mínimo	Admin	Baja	Backlog

5. Lecciones Aprendidas y Conclusiones

Logros del equipo

- Se implementaron las **3 User Stories del rol Vendedor** en su totalidad: generación de cupones, asignación a clientes con notificación por email y módulo completo de reportes analíticos.
- El **MVP obligatorio de la cátedra** quedó completamente implementado al cierre del Sprint 2, liberando el Sprint 3 para las funcionalidades del Vendedor y el pulido visual.
- La **arquitectura de componentes** permitió que cada integrante trabajara en paralelo sin conflictos: Genaro en la infraestructura/App.tsx, Paulo en los componentes del cliente y Maxi en los reportes.
- La **lógica de negocio** centralizada en helpers.ts (validateCoupon, simulateSendEmail) con TypeScript strict facilitó el testing manual y la detección temprana de errores.

Desafíos encontrados

- **Sincronización de estado entre portales:** al elevar el estado a App.tsx, los re-renders afectaban componentes no relacionados. Se mitigó con keys explícitas y callbacks estables.
- **Decisión de posponer el backend:** inicialmente planificado desde el Sprint 1, se postergó para priorizar el MVP funcional en el frontend. Decisión correcta pero genera deuda técnica.
- **Coordinación en equipo de 3 personas:** la gestión en Jira y las ramas de GitHub fueron fundamentales para no pisar el trabajo ajeno, especialmente en la integración del InteractiveSimulator con los cupones.

Conclusiones

El proyecto demostró que un enfoque iterativo e incremental con Scrum permite entregar valor tangible en cada sprint, ajustando el alcance según el feedback del Product Owner. La decisión de empezar con un frontend standalone con datos en memoria permitió validar la lógica de negocio rápidamente sin bloqueos de infraestructura. Las próximas iteraciones deberían priorizar la integración del backend Spring Boot para habilitar persistencia real, emails transaccionales y la API de Mercado Pago.

6. Integrantes del Grupo

Nombre y Apellido	Rol en el Equipo	Mail	Legajo
Genaro Molina	Scrum Master / Full Stack	genamolina01@gmail.com	17207
Maximiliano Zampa	Development Team / Frontend	maximilianopablo14@gmail.com	17442
Paulo Zito	Development Team / Frontend	paulozt5@gmail.com	15552